

DESIGN & CALCULATIONS**Truth Table:**

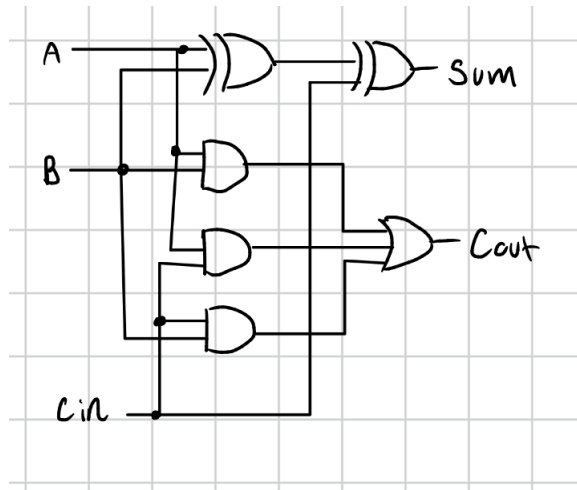
Inputs			Outputs	
Cin	B	A	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Logic Equations:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

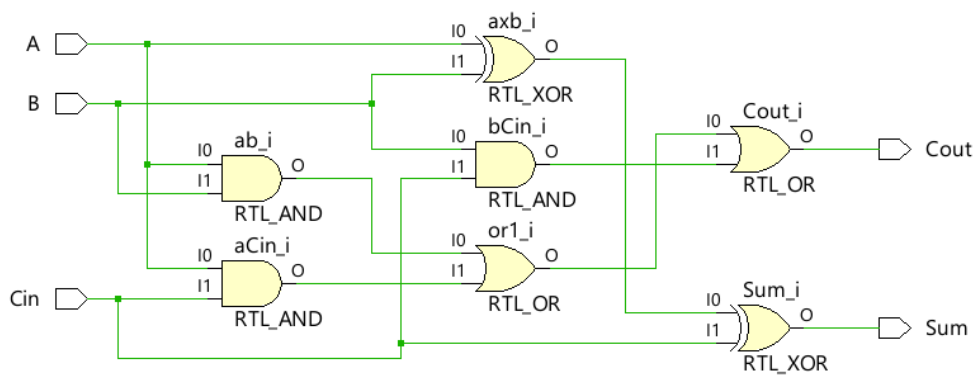
$$\text{Cout} = (A \cdot B) + (A \cdot \text{Cin}) + (B \cdot \text{Cin})$$

Circuit/ Block Diagrams:



My Sketch:

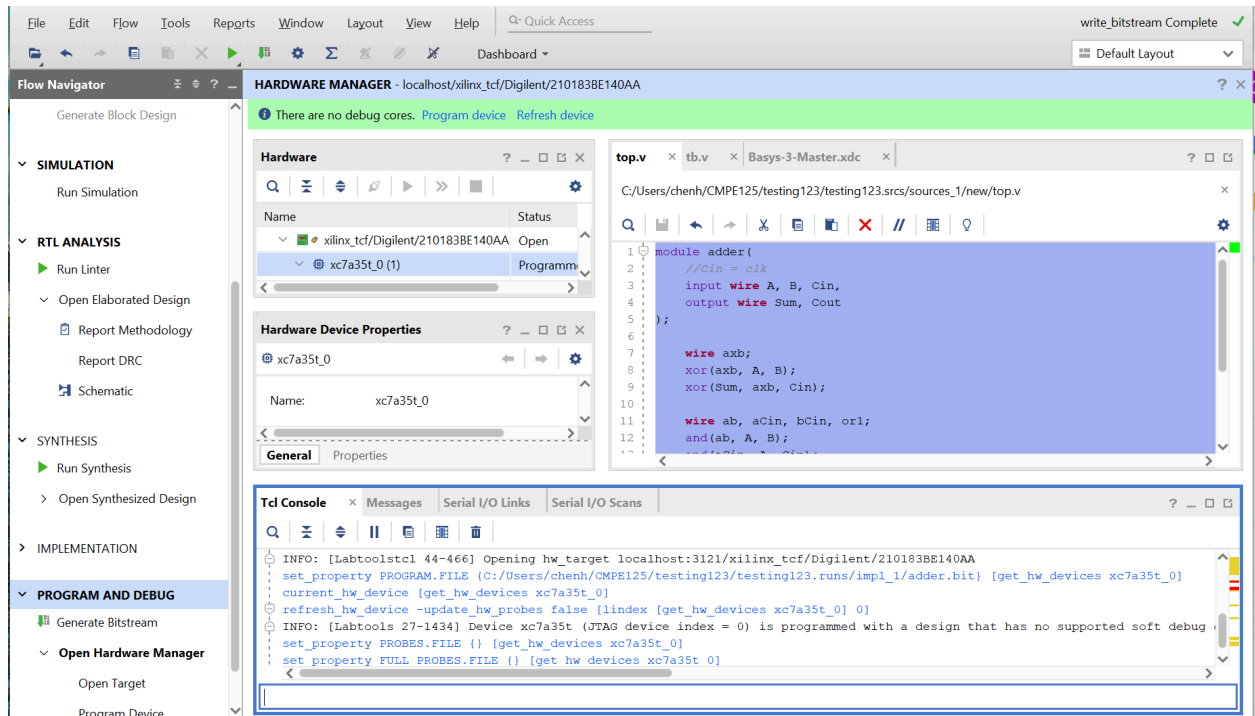
Vivado Schematic:



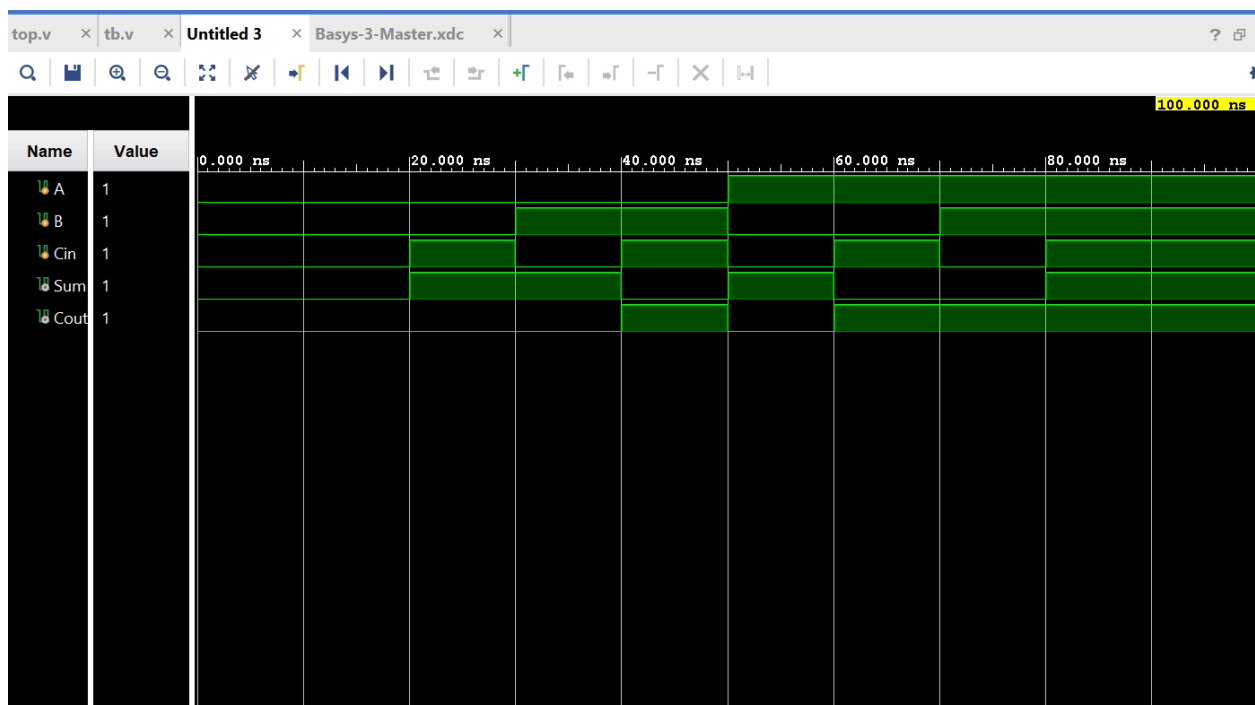
SCREENSHOTS

Compilation:

Successful Synthesis, Implementation, Bitstream → Hardware Manager



Simulation Waveforms:



SUMMARY

The adder has 3 inputs: A, B, and Cin, which represent bits that get added to produce two binary digits: Cout (the MSB) and Sum (LSB). Thus, when all three inputs are on, the binary digits

should be: 11, where Cout=Sum=1. Refer to the truth table for the full list of expected outputs. I wrote the source code for the adder module as well as a testbench that runs through all 8 combinations of inputs for A, B, and Cin. I uncommented some lines and changed some variable names of 3 switches and 2 LEDs on the constraint file so my FPGA board can be programmed accordingly.

VERILOG CODE

Adder.v: (gate implementation)

```
module adder(
    input wire A, B, Cin,
    output wire Sum, Cout
);

    wire axb;
    xor(axb, A, B);
    xor(Sum, axb, Cin);

    wire ab, aCin, bCin, or1;
    and(ab, A, B);
    and(aCin, A, Cin);
    and(bCin, B, Cin);
    or(or1, ab, aCin);
    or(Cout, or1, bCin);

Endmodule
```

Adder.v: (high level implementation)

```
module adder(
    input wire A, B, Cin,
    output wire Sum, Cout
);

    wire axb, ab, aCin, bCin;
    assign axb = A ^ B;
    assign Cout = axb ^ Cin;
    assign ab = A*B;
    assign aCin = A*Cin;
    assign bCin = B*Cin;
    assign Sum = ab + aCin + bCin;
```

```

        endmodule

tb.v:
`timescale 1ns/1ps
module tb;
reg A, B, Cin;
wire Sum, Cout;

    adder out(.A(A), .B(B),.Cin(Cin), .Cout(Cout), .Sum(Sum));

    initial begin
        A=0; B=0; Cin=0;

        $monitor("Time=%0t | A=%b B=%b Cin=%b | Cout=%b Sum=%b",
            $time, A, B, Cin, Cout, Sum);

        #10 A=0; B=0; Cin=0;
        #10 A=0; B=0; Cin=1;
        #10 A=0; B=1; Cin=0;
        #10 A=0; B=1; Cin=1;
        #10 A=1; B=0; Cin=0;
        #10 A=1; B=0; Cin=1;
        #10 A=1; B=1; Cin=0;
        #10 A=1; B=1; Cin=1;

        #20 $finish;
    end

endmodule

```

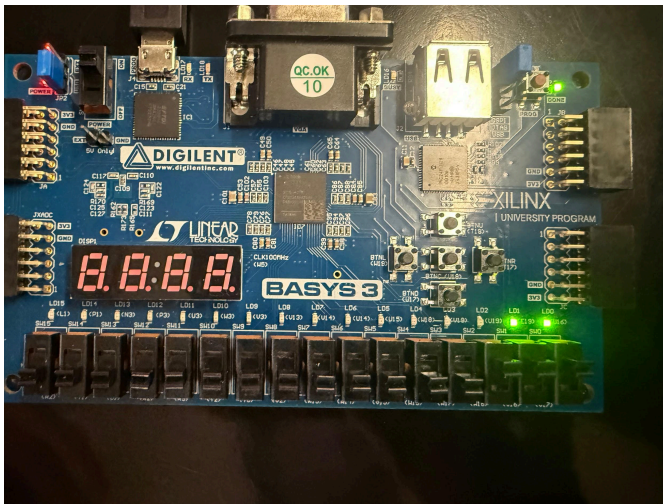
TROUBLESHOOTING

I mislabeled some variable names on the constraint file, so my code did not compile successfully—we must keep in mind for future labs that Verilog, like C, is case sensitive. Also, at first I implemented the design module using gate-logic, but switched to behavioral logic after.

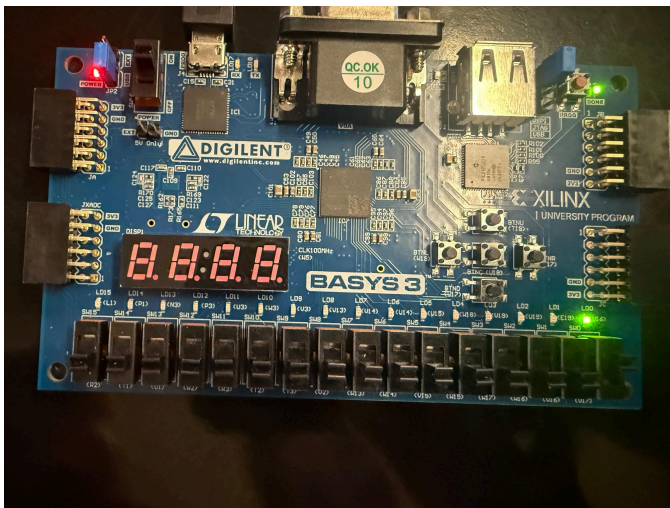
FPGA BOARD OUTPUT

Variables:

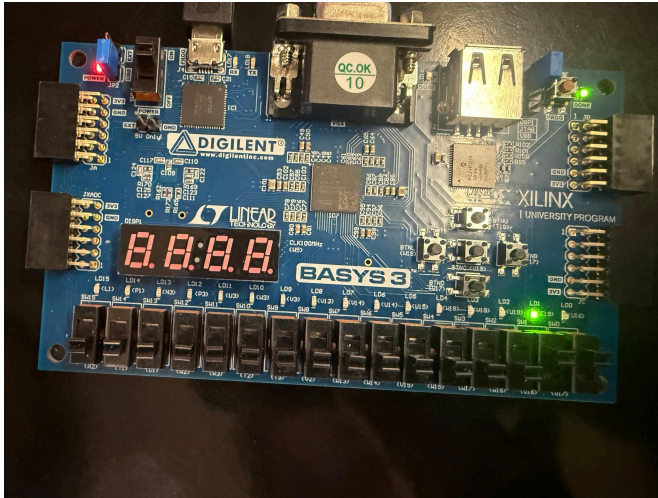
- Sw2= A
- Sw1=B
- Sw0=Cin
- Led0=Sum
- Led1=Cout



A=1, B=1, Cin=1 $\rightarrow$ Cout=1, Sum=1



A=0, B=0, Cin=1 $\rightarrow$ Cout=0, Sum=1



A=0, B=1, Cin=1 $\rightarrow$ Cout=1, Sum=0